

Recovering coordinates from paper maps

Gabriel Arellano

2026-09-16

Table of contents

1	Purpose	1
2	Setup	2
3	Recommended method: homography	2
4	Plan B: affine transformation	5
5	Obtaining clicked coordinates	6
6	Local and global plot coordinates	7
7	Historical CTFS compatibility only	8

1 Purpose

Points clicked on a scanned or photographed map are recorded in image coordinates. Control points connect that image to known plot coordinates. The recommended transformation is a **homography**, using the four corners of the mapped quadrat. It corrects photographic perspective and performed almost as well as the affine transformation on flatbed scans in the [coordinate-recovery evaluation](#). Use an **affine transformation** as a fallback for a flat or nearly flat image when one corner is missing or a corner click is unusually unreliable.

These transformations correct one whole planar image. They do not repair folds, tears, uneven stretching or errors in the original field map.

2 Setup

Run this tutorial inside a clone of the fgeo2 repository. The code uses only base R and the example data stored in the repository.

```
p <- system2("git", c("rev-parse", "--show-toplevel"), stdout = TRUE)
if(!is.null(attr(p, "status")) || length(p) != 1L)
  stop("Run this tutorial inside the fgeo2 Git repository.")
p <- normalizePath(p)

source(file.path(
  p, "06_topography_habitat", "R", "new", "georeference.R"
))

example.path <- file.path(
  p, "data_examples", "coordinate_recovery_test"
)
```

3 Recommended method: homography

Case A is an untreated paper map digitized with a flatbed scanner. The file contains four clicked corners followed by clicked test points. The four known corners describe a 10×10 m quadrat in global plot coordinates.

```
clicked <- read.csv(file.path(example.path, "points test A.csv"))

known.controls <- data.frame(
  id = paste0("corner_", LETTERS[1:4]),
  x = c(50, 50, 60, 60),
  y = c(50, 60, 60, 50)
)

# Match controls by identifier; do not rely on file row order.
control.order <- match(known.controls$id, clicked$id)
stopifnot(!anyNA(control.order))

known.xy.of.controls <- as.matrix(known.controls[, c("x", "y")])
clicked.xy.of.controls <- as.matrix(
  clicked[control.order, c("x", "y")]
)
rownames(known.xy.of.controls) <- known.controls$id
```

```

rownames(clicked.xy.of.controls) <- clicked$id[control.order]
stopifnot(identical(
  rownames(known.xy.of.controls),
  rownames(clicked.xy.of.controls)
))

is.control <- clicked$id %in% known.controls$id
clicked.xy.of.points <- as.matrix(clicked[!is.control, c("x", "y")])
rownames(clicked.xy.of.points) <- clicked$id[!is.control]

```

The result uses the units and coordinate system of `known.xy.of.controls`. Here it is returned directly in global plot coordinates and metres.

```

homography.xy <- apply_homography_transformation(
  known.xy.of.controls = known.xy.of.controls,
  clicked.xy.of.controls = clicked.xy.of.controls,
  clicked.xy.of.points = clicked.xy.of.points
)

recovered <- data.frame(
  id = rownames(homography.xy),
  global.x = homography.xy[, "x"],
  global.y = homography.xy[, "y"],
  row.names = NULL
)

head(recovered)

```

```

  id global.x global.y
1 p1 47.95860 47.94644
2 p2 49.96120 47.98412
3 p3 52.04244 47.93259
4 p4 53.97912 47.96596
5 p5 55.97250 47.96855
6 p6 57.97141 47.95747

```

The example was constructed from known points, so it also provides a simple check of the result.

```

known.points <- expand.grid(
  x = seq(48, 62, by = 2),
  y = seq(48, 62, by = 2)
)
known.points$id <- paste0("p", seq_len(nrow(known.points)))
known.order <- match(recovered$id, known.points$id)
stopifnot(!anyNA(known.order))

error.cm <- 100 * sqrt(
  (recovered$global.x - known.points$x[known.order])^2 +
  (recovered$global.y - known.points$y[known.order])^2
)

round(c(
  mean.error.cm = mean(error.cm),
  median.error.cm = median(error.cm),
  maximum.error.cm = max(error.cm)
), 1)

```

mean.error.cm	median.error.cm	maximum.error.cm
3.5	3.3	8.0

```

plot(
  recovered$global.x,
  recovered$global.y,
  asp = 1,
  xlab = "Global plot x (m)",
  ylab = "Global plot y (m)",
  pch = 20
)
polygon(known.controls$x, known.controls$y, border = "red", lwd = 2)

```

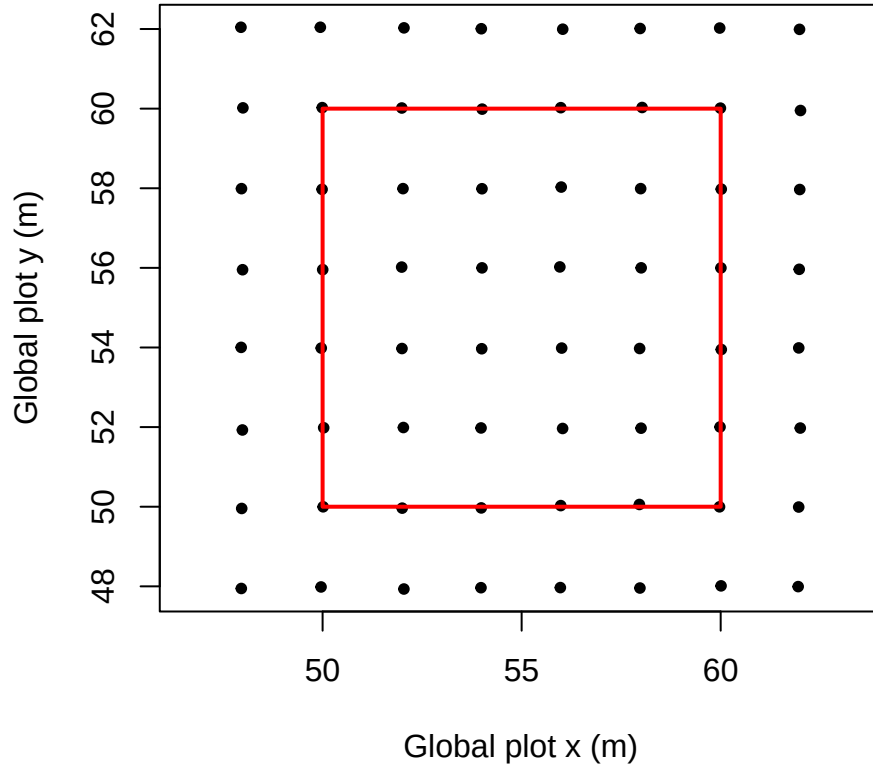


Figure 1: Coordinates recovered from test case A. The square marks the four control locations.

4 Plan B: affine transformation

An affine transformation needs at least three non-collinear controls. It is a practical fallback for a flatbed scan when one corner is unavailable. Do not use three controls to correct appreciable photographic perspective. If four or more reliable controls are available, retain them: the affine fit uses all of them by least squares.

This example deliberately omits corner D from the same scan.

```
available.controls <- paste0("corner_", LETTERS[1:3])
available <- match(available.controls, known.controls$id)

affine.xy <- apply_affine_transformation(
  known.xy.of.controls = known.xy.of.controls[available, ],
  clicked.xy.of.controls = clicked.xy.of.controls[available, ],
  clicked.xy.of.points = clicked.xy.of.points
)
```

```
affine.recovered <- data.frame(
  id = rownames(affine.xy),
  global.x = affine.xy[, "x"],
  global.y = affine.xy[, "y"],
  row.names = NULL
)

head(affine.recovered)
```

```
   id global.x global.y
1 p1 47.96200 47.94982
2 p2 49.96124 47.97892
3 p3 52.04204 47.91816
4 p4 53.98110 47.94325
5 p5 55.97977 47.93716
6 p6 57.98690 47.91730
```

5 Obtaining clicked coordinates

The transformations do not depend on ImageJ. Any tool that exports matching point identifiers and image x-y coordinates can be used. In R, the minimal interactive approach is `locator()`; see the complete [example script](#). Click the four corners in a stated order, then the mapped points, and press Esc or **Finish** when done. The direction of the image y-axis does not matter if every corner is identified correctly.

```
pdf.path <- file.path(example.path, "test A.pdf")
map.image <- magick::image_read(pdf.path, density = "300x300")[1]
map.raster <- as.raster(map.image)
plot(map.raster, interpolate = FALSE)

clicked <- locator(n = 1000, type = "p", pch = 19, col = "red")
clicked <- data.frame(x = clicked$x, y = clicked$y)
clicked$id <- c(
  paste0("corner_", LETTERS[1:4]),
  paste0("point_", seq_len(nrow(clicked) - 4L))
)
clicked <- clicked[, c("id", "x", "y")]
write.csv(clicked, "clicked-points.csv", row.names = FALSE)
```

ImageJ or [Fiji](#) can also export the points. The historical workflow used a modified [PointPicker plugin](#). Point collection and coordinate recovery are separate steps; ImageJ is not required for either transformation above.

6 Local and global plot coordinates

The known controls define the output coordinates. Corners such as (50, 50), (50, 60), (60, 60) and (60, 50) return global plot coordinates directly. Corners entered as (0, 0), (0, 10), (10, 10) and (10, 0) return coordinates local to that 10 x 10 m quadrat.

If a process must use local controls, convert its results afterwards with `compile_points_from_quadrats()`. The function and a fuller explanation are in the [quadrat source](#) and [quadrat tutorial](#).

```
source(file.path(p, "03_spatial", "R", "new", "quadrats.R"))
```

```
local.points <- data.frame(
  id = c("tree_1", "tree_2"),
  x = c(2.1, 8.4),
  y = c(3.7, 6.2)
)
```

```
quadrat.locations <- data.frame(
  quadrat_id = "q_50_50",
  xmin = 50,
  ymin = 50,
  xmax = 60,
  ymax = 60
)
```

```
global.points <- compile_points_from_quadrats(
  inner.x = local.points$x,
  inner.y = local.points$y,
  point.id = local.points$id,
  quadrat.id = rep("q_50_50", nrow(local.points)),
  quadrat.locations = quadrat.locations
)
```

```
global.points
```

	point_id	outer.x	outer.y
1	tree_1	52.1	53.7
2	tree_2	58.4	56.2

These are plot-coordinate transformations, not conversions between geographic coordinate reference systems.

7 Historical CTFS compatibility only

The CTFS/ImageJ conversion is not the recommended pipeline and will not be retained in fgeo2. The archived [imageJ.r implementation](#) remains available with its [geometry dependencies](#). The original workflow is documented in the 2011 CTFS [PDF tutorial](#) and [HTML tutorial](#).

The archived `distance.to.side()` passes one-column data frames instead of numeric intersection coordinates. If the old workflow is still required, run the following session-only replacement after loading the archived CTFS scripts. It fixes that one bug only. It does not fix inconsistent corner defaults, exterior points folded into a quadrat, or other limitations of the legacy workflow. Supply all four corner labels explicitly when calling the old functions.

```
condit.path <- file.path(p, "evaluation", "Condit")
source(file.path(condit.path, "startCTFS", "startCTFS.r"))
startCTFS(condit.path)

distance.to.side.original <- distance.to.side

distance.to.side <- function(pt, target, side1, side2)
{
  perpendicular <- perpendicular.distance(
    b = target[1], m = target[2], x = pt[1], y = pt[2]
  )

  parallel <- drop(as.matrix(parallel.line(
    b = side1[1], m = side1[2], x = pt[1], y = pt[2]
  )))
  intersection <- intersection.of.lines(
    b1 = parallel[1], m1 = parallel[2],
    b2 = target[1], m2 = target[2]
  )
  distance1 <- xydist(pt[1], pt[2], intersection$x, intersection$y)

  parallel <- drop(as.matrix(parallel.line(
    b = side2[1], m = side2[2], x = pt[1], y = pt[2]
  )))
  intersection <- intersection.of.lines(
    b1 = parallel[1], m1 = parallel[2],
```

```

    b2 = target[1], m2 = target[2]
  )
  distance2 <- xydist(pt[1], pt[2], intersection$x, intersection$y)

  distances <- c(perpendicular, distance1, distance2)
  c(mean(distances), sd(distances) / mean(distances))
}

stopifnot(isTRUE(all.equal(
  unname(distance.to.side(
    pt = c(5, 5),
    target = c(0, Inf),
    side1 = c(10, 0),
    side2 = c(0, 0)
  )),
  c(5, 0)
)))

```